# Как читать вывод тестов в PHP

Полезно при прохождении любых курсов по PHP

Чтобы разобраться в выводе тестов, в первую очередь разберемся, как работают тесты на платформе.

Тестирование кода на PHP происходит с помощью фреймворка PHPUnit, который тоже написан на PHP. Проверка кода выполняется автоматически при нажатии на кнопку “Проверить”, после чего тестируемая программа загружается в память и вызывается с разными параметрами, а тесты следят за тем, чтобы ее поведение соответствовало ожидаемому.

После этого на вкладке OUTPUT можно увидеть результат проверки. И вот здесь, если возникают ошибки, могут быть трудности с пониманием того, что происходит. Здесь нам поможет чтение вывода тестов — нашего главного помощника в отладке кода.

## Виды ошибок

Никто не пишет код идеально. Поэтому рано или поздно вам придется столкнуться с ошибками. И понимание, какого вида ошибка, поможет вам разобраться в корне проблемы.

Условно ошибки, возникающие при выполнении упражнения, можно разделить на два вида:

- ошибки интерпретатора — разного рода ошибки и исключения, встроенные в PHP.
- ошибки утверждений (AssertionError) — вызванные тестами. Результат работы функции не соответствует ожидаемому результату.

### Ошибки интерпретатора

Ждем ваши вопросы!

Ошибки интерпретатора можно условно разделить на две категории:

- синтаксические ошибки
- семантические ошибки, которые чаще называют исключениями.

**Синтаксические ошибки (SyntaxError)** — это нарушение синтаксиса и пунктуации языка. Во время запуска интерпретатор PHP не понимает, как прочесть ошибочное выражение, и вызывает ошибку **SyntaxError**. Пропустили знак доллара перед именем переменной — интерпретатор покажет ошибку.

```
phpunit --colors=always tests
PHPUnit 9.5.16 by Sebastian Bergmann and contributors.

Error in bootstrap script: ParseError:
syntax error, unexpected token "="
```

**Семантические ошибки** — не препятствуют запуску программы, однако программа не работает как ожидается. В PHP существует огромное количество ошибок и исключений, но знать их не обязательно — чаще всего смысл ошибки скрыт в ней самой или же в названии выброшенного исключения. Например, при обращении к необъявленной переменной возникнет ошибка `Undefined variable`, которая дословно говорит о том, что переменная `$sum` не определена.

```
There was 1 error:

1) App\Tests\SolutionTest::testSumOfSeries
Undefined variable $sum
```

Не стоит бояться подобных ошибок: достаточно прочитать описание и станет понятно, что хочет интерпретатор. Если даже из описания не ясно, что требуется, то всегда есть возможность обратиться к документации, в которой описаны случаи возникновения тех или иных ошибок.

Важно понять, что данные ошибки не связаны с тестами и сигнализируют о проблеме в коде.

### Утверждения

Отдельно стоит выделить ошибки утверждений. Данные ошибки возникают в случае несоответствия ожидаемого значения с результатом работы функции.

Для начала разберемся, как выглядят тесты для упражнений:

```php
public function testSumOfSeries()
{
    assert(6 === sumOfSeries(6, 6));
    assert(3 === sumOfSeries(1, 2));
    assert(24 === sumOfSeries(7, 9));
}
```

Тестируемая функция, в данном случае `sumOfSeries()`, вызывается с определенными параметрами, и результат ее работы сравнивается с заранее определенным ожидаемым результатом.

Однако в случае, если значения не совпадают, возникает ошибка. Рассмотрим подробнее вывод на вкладке OUTPUT в случае падения тестов:

```
There was 1 failure:

1) App\Tests\SolutionTest::testSumOfSeries
Assertion failed 6 === sumOfSeries(6,6) -> false
# sumOfSeries(6,6) -> 1

/usr/src/app/tests/SolutionTest.php:13
```

Здесь мы видим, что выполнение остановилось на тесте под названием testSumOfSeries. Далее говорится, что сравнение ожидаемого результата, а это цифра 6, и результата работы функции вернуло `false`. Для того чтобы этот тест-кейс считался пройденным, сравнение должно вернуть `true`.

На следующей строке мы видим фактический результат вызова `sumOfSeries(6, 6)`, который равен 1.

Наконец, на последней строке выводится путь до файла с тестами с указанием через: номера строки, на которой остановилось выполнение.

После этого нужно выяснить, чем конкретно вызвано несовпадение результатов, и устранить причину неправильной работы тестируемой функции.

Взглянем на еще один вариант вывода тестов, который может встретиться вам в курсах:

```
1) App\Tests\ArraysTest::testGetSameParity
Failed asserting that two arrays are equal.
--- Expected
+++ Actual
@@ @@
 Array (
     0 => 1
-    1 => 3
+    1 => 1
 )

/usr/src/app/tests/ArraysTest.php:16
```

Он очень похож на предыдущий, но здесь сравниваются два массива. Ожидаемый и фактически полученный при вызове функции.

- со знаком «`-`» указывается ожидаемый результат
- со знаком «`+`» указывается полученный результат, который не ожидался
- без отметки – результат, который совпадает с ожидаемым

По выводу выше мы можем сказать, что был получен массив `[1, 1]`, а должен был `[1, 3]`.

### Отладка кода

Иногда с первого взгляда невозможно понять, чем вызвано неправильное поведение функции. Тогда на помощь приходит отладка — возможность заглянуть в процесс выполнения кода. Существует несколько способов, но мы рассмотрим самый простой и распространенный — отладочную печать.

Отладочная печать — вывод на экран значений переменных, вызовов функций, сообщений о выполнении итерации цикла в процессе работы программы. Для этого достаточно добавить вызов функции `print_r()` или `var_dump()` с интересующим нас значением.

Вывод этих функций, хоть они и встроены в PHP, часто оставляет желать лучшего. Чтобы получить красиво отформатированный вывод во всех практиках на Хекслете подключена функция `dump()` из библиотеки VarDumper. Эта функция также принимает в качестве параметра значение, которое нужно вывести на экран.

Посмотрим как она работает. Добавим вызов функции dump() в код.

```
$result = '';
for ($i = 0; $i < $len; $i++) {
    dump($result);
    $symbol = mb_substr($text, $i, 1);
```

После этого достаточно запустить проверку тестами еще раз, чтобы на вкладке OUTPUT увидеть результат работы нашей отладочной печати:




С помощью отладочной печати становится легче следить за процессом выполнения программы, а значит, установить, куда закралась ошибка, становится намного проще. Подробнее вопрос отладки рассмотрен в уроке “Отладка" курса “Основы PHP”.

👆 На этом пока всё